INFO-532
Database Systems Homework 3 Part 1

(Friday March 07, 2025, at Midnight to Brightspace)

No late submission will be accepted

Please remember to include the following statement at the beginning of your submitted assignment and SIGN it. Your assignment won't be graded with the signed statement.
"I have done this assignment completely on my own. I have not copied it, nor have I given my solution to anyone else. I understand that if I am involved in plagiarism or cheating, I will have to sign an official form that I have cheated and that this form will be stored in my official university record. I also understand that I will receive a grade of 0 for the involved assignment and my grade will be reduced by one level (e.g., from A to A- or from B+ to B) for my first offense, and that I will receive a grade of "F" for the course for any additional offense of any kind."

1. (22 points) The following are some of the relations transformed from the ER diagram for the Student Registration System (note that some changes have been made due to the creation of a single-attribute primary key for Classes):

   Students(B#, first_name, last_name, level, gpa, email, bdate)
   Courses(dept_code, course#, title, credits, deptname)
   Classes(classid, dept_code, course#, sect#, year, semester, start_time, end_time, limit, size, room, Faculty_B#, TA_B#) /*note: classid is added to serve as a single-attribute primary key */
   Faculty(B#, first_name, last_name, title, office, email, phone#, deptname)
   G_Enrollment(G_B#, classid, lgrade, score)

   Do the following for each relation schema, identify all non-trivial functional dependencies based on the Requirements Document (but take into consideration that classid is added as the primary key for Classes). For this question, we also make the following assumptions: (1) each dept_code corresponds to a unique department and vice versa; (2) only faculty members in the same department could share an office and a phone number; (3) each faculty office (shared or not) has one phone with a unique number. Don't make other assumptions about the data. Use the union rule to combine the functional dependencies as much as possible to avoid having multiple functional dependencies with the same left-hand side but different right-hand side. Furthermore, try not to list redundant FDs. For example, if you already have A → BC, you don't need to also list A → B and A → C. As another example, if you already have A → B and B → C, you don't need to include A → C. But you are not required to eliminate all redundant FDs at this time.

2. Consider the following table schema for accounts in a banking system:

   Accounts(acct#, owner_id, owner_name, acct_type, balance, interest_rate, time_opened)

   It has the following functional dependencies F = {acct# → own_id acct_type balance time_opened, owner_id → owner_name, acct_type balance → interest_rate}.

   (1) (5 points) Explain why the schema is not in BCNF.
   (2) (14 points) Use Algorithm lossless-join-decomposition BCNF (LLJD-BCNF) to decompose it. Show the steps.

(3) (5 points) Is your decomposition dependency-preserving? Justify your answer.

3. (10 points) Disprove the following rule:

   $\{B \rightarrow CD, AB \rightarrow E, E \rightarrow C\} \models \{AE \rightarrow CD\}$

   To disprove a rule, construct a relation with attributes (A, B, C, D, E) and some tuples such that the tuples of the relation satisfy the functional dependencies on the left-hand side of the rule (left of |=) but do not satisfy the functional dependency on the right-hand side of the rule.